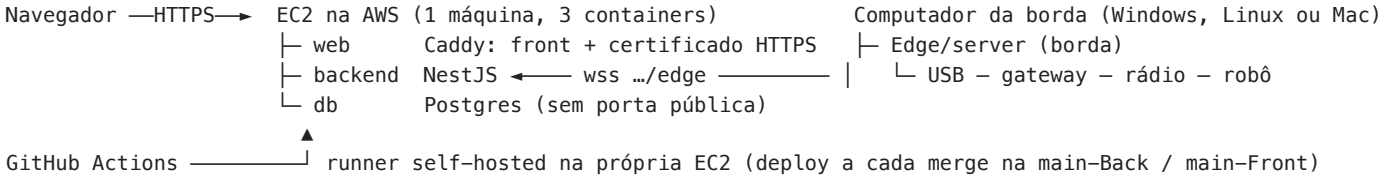
Deploy na AWS (Learner Lab): ambiente novo do zero

Tutorial completo para qualquer pessoa montar o servidor, ligar a borda (robô real) e configurar o GitHub com as pipelines de verificação e deploy.



- **Uma máquina só (EC2)** com Docker. É o mesmo `docker compose` do dev, sem serviço gerenciado.
- **Endereço grátis com HTTPS**: o sslip.io transforma o IP em nome (`54-12-34-56.sslip.io` → `54.12.34.56`) e o Caddy tira o certificado sozinho.
  - Front: `https://54-12-34-56.sslip.io`
  - API: `https://api.54-12-34-56.sslip.io` (Swagger em `/api`)
- **A borda roda em qualquer computador** (Windows, Linux ou Mac) com internet e uma porta USB para o gateway, e é **ela que conecta no back** (`wss://api.<domínio>/edge` , com o `EDGE_TOKEN`), como um navegador abrindo um site. Funciona em qualquer rede com internet, sem abrir porta nesse computador e sem VPN. Telemetria e comandos passam por essa conexão.
- **Deploy automático e separado**: merge na `main-Back` sobe só o back; merge na `main-Front` sobe só o front. A `main` não faz deploy.

Roteiro

Parte	Passos	Onde
A. Servidor	1 a 9: criar a EC2, IP fixo, baixar o código, preparar, conferir o `.env` , subir, primeiro usuário	Console da AWS + terminal da EC2
B. Borda	10: preparar o computador da borda e ligar no back	Computador da borda (Windows, Linux ou Mac)
C. GitHub	11 a 15: runner na EC2, segurança do Actions, regras das branches, variáveis, primeiro deploy	GitHub + terminal da EC2
D. Referência	como a pipeline funciona, dia a dia, banco, problemas comuns, segurança, outro ambiente	—

Arquivos envolvidos

Arquivo	Branch	Para quê
`deploy/docker-compose.yml`	back	Sobe `db`, `backend` e `web`
`deploy/Caddyfile`	back	Domínio, HTTPS, rotas da SPA e proxy da API
`deploy/.env.example`	back	Modelo das variáveis do servidor
`deploy/setup-ec2.sh`	back	Prepara a EC2 do zero (Docker, swap, `.env`)
`Backend/server/Dockerfile`	back	Imagem da API
`Frontend/Dockerfile`	front	Build do Vite servido pelo Caddy
`.github/workflows/backVerify.yml`, `mainBackRule.yml`, `prodBackDeploy.yml`	back	Verificação no PR para `main-Back` e deploy do back
`.github/workflows/frontVerify.yml`, `mainFrontRule.yml`, `prodFrontDeploy.yml`	front	Verificação no PR para `main-Front` e deploy do front
`Edge/server` com uplink (`BACKEND_URL` no `.env.example`)	borda	Borda que conecta no back

Limites do Learner Lab que importam

- **Região: só `us-east-1` (N. Virginia).** Use essa em tudo.
- **Só imagens da Amazon**: use **Amazon Linux 2023** (o padrão).
- **A sessão do lab acaba** (o contador fica na página do lab) e a AWS **desliga** a EC2. Na próxima sessão você liga de novo (ver "Dia a dia"). Dados, site e runner voltam sozinhos.
- **Créditos**: `t3.small` ≈ US$ 0,02/h + IP elástico ≈ US$ 0,005/h + disco 20 GB ≈ US$ 1,60/mês. O orçamento aparece na página do lab.
- **Nunca "Encerrar" (Terminate)** a instância: apaga a máquina e o banco.

Antes de começar

1. **Acessos**: AWS Academy com o Learner Lab; no GitHub, ser **admin** do repositório (precisa ver a aba **Settings**); um computador para a borda (Windows, Linux ou macOS) com internet, uma porta USB livre e o gateway (nRF5340 DK) já gravado.
2. **Branches prontas**: a `main-Back` e a `main-Front` precisam ter os arquivos da tabela acima. O front tem que passar no `npm run build` (em `Frontend/`, no seu computador): é o mesmo comando do servidor e da pipeline, e um `error TS...` aí derruba o deploy do front.
3. **Repositório privado?** Crie um token no GitHub (Settings da sua conta → Developer settings → Personal access tokens → *Fine-grained*, só este repo, *Contents: Read-only*). Ele é a "senha" no `git clone`. Repositório público não precisa.

Parte A: servidor

O console da AWS pode estar em português ou em inglês; os nomes aparecem nos dois idiomas abaixo.

Passo 1: abrir o console da AWS

1. AWS Academy → seu curso → **Modules** → **Learner Lab**.
2. **Start Lab** e espere a bolinha ao lado de "AWS" ficar **verde**.
3. Clique em **AWS** (abre o console).
4. No canto superior direito, confira a região: **N. Virginia (us-east-1)**.

Passo 2: criar a máquina (EC2)

Na busca do console digite **EC2** → **Executar instância** (*Launch instance*):

Campo	Valor
Nome (*Name*)	`tcc-server`
Imagens de aplicação e de sistema operacional (*Application and OS Images*)	**Amazon Linux 2023** (já vem selecionado)
Tipo de instância (*Instance type*)	**t3.small** (a `t3.micro` tem pouca memória para o build)
Par de chaves (*Key pair*)	**vockey**
Configurações de rede (*Network settings*)	ver abaixo
Configurar armazenamento (*Configure storage*)	**20** GiB, **gp3**
Detalhes avançados → Perfil de instância do IAM (*IAM instance profile*)	**LabInstanceProfile** (plano B de terminal, passo 4)

Configurações de rede:

- **Atribuir IP público automaticamente** (*Auto-assign public IP*): deixe **Habilitar**. Sem ele a máquina fica sem internet até o passo 3.
- **Firewall**: **Criar grupo de segurança** (*Create security group*), com três regras de entrada, todas com origem **Qualquer lugar** (`0.0.0.0/0`):
  - **SSH**, porta 22 (terminal pelo navegador)
  - **HTTP**, porta 80 (o Caddy usa para tirar o certificado)
  - **HTTPS**, porta 443 (o site e a API)

  A tela pode aparecer de dois jeitos:

  - **Caixas de marcar**: marque *Permitir tráfego SSH de / HTTPS da internet / HTTP da internet* (*Allow SSH / HTTPS / HTTP traffic*).
  - **Lista de regras** (depois de clicar em **Editar**): a regra 1 já vem com SSH; clique em **Adicionar regra de grupo de segurança** duas vezes e escolha **HTTP** e **HTTPS** em *Tipo*, com *Tipo de origem* **Qualquer lugar**.

  Esqueceu? Dá para arrumar depois: instância → aba **Segurança** → link do grupo (`sg-...`) → **Editar regras de entrada**.

Executar instância → **Visualizar todas as instâncias**. Espere o *Estado da instância* ficar **Executando** (*Running*).

Passo 3: IP fixo (IP elástico)

Sem isso o IP muda toda vez que o lab reinicia, e o endereço do site muda junto.

1. Menu da esquerda: **Rede e segurança → IPs elásticos** (*Elastic IPs*) → **Alocar endereço IP elástico** → **Alocar**.
2. Selecione o IP criado → **Ações → Associar endereço IP elástico** → *Instância*: `tcc-server` → **Associar**.
3. Confira em **Instâncias →** **`tcc-server`** : o **Endereço IPv4 público** tem que ser o IP elástico.
4. Anote o domínio: IP `54.12.34.56` → `54-12-34-56.sslip.io` (pontos viram traços).

> O `172.31.x.x` que aparece no terminal (`ip-172-31-...`) é o IP interno da máquina, não o público.

Passo 4: abrir o terminal da máquina

Instâncias → marque `tcc-server` → **Conectar** → aba **EC2 Instance Connect** → **Conectar**.

Abre um terminal no navegador. Todos os comandos da Parte A são colados nele (colar: `Ctrl+V` ou `Ctrl+Shift+V` no Windows/Linux, `Cmd+V` no Mac).

> Não conectou? Mesma tela, aba **Session Manager** → **Conectar**, e rode `sudo su - ec2-user` antes de tudo.

Passo 5: baixar o código

O servidor usa **dois clones**, um para cada lado, porque o deploy automático atualiza cada um na sua branch sem mexer no outro:

```
sudo dnf install -y git
mkdir -p ~/tcc && cd ~/tcc
git clone -b main-Back  https://github.com/Lucas-O-S/TCC_Swarm_Robots.git back
git clone -b main-Front https://github.com/Lucas-O-S/TCC_Swarm_Robots.git front
```

Repo privado: *Username* = seu usuário do GitHub, *Password* = o token (não aparece nada enquanto você cola, é normal).

Passo 6: preparar a máquina (um comando)

```
bash ~/tcc/back/deploy/setup-ec2.sh
```

Leva uns 2 a 3 minutos e pode ser rodado de novo quantas vezes quiser (pula o que já está feito e nunca sobrescreve o `.env`). Ele:

1. confere se a máquina é Amazon Linux;
2. cria 2 GB de **swap** (memória extra para o build não travar a `t3.small`);
3. instala Docker, git e openssl, e liga o Docker junto com a máquina;
4. instala os plugins **Docker Compose** e **Buildx** (o Buildx do Amazon Linux é antigo demais);
5. cria o `deploy/.env` com o `DOMAIN` tirado do IP público e senhas aleatórias para o banco, o `JWT_SECRET` e o `EDGE_TOKEN` .

No fim ele mostra o `DOMAIN` : **confira se é o IP elástico do passo 3**.

Passo 7: conferir o `.env`

```
nano ~/tcc/back/deploy/.env
```

(Salvar e sair do `nano` : `Ctrl+O` , `Enter` , `Ctrl+X` .)

Variável	O que é	Valor
`DOMAIN`	Endereço do site	IP elástico com traços + `.sslip.io`
`FRONT_DIR`	Onde está o clone do front, a partir de `deploy/`	`../../front`
`DB_USERNAME` , `DB_PASSWORD` , `DB_NAME`	Banco	gerados; ver aviso abaixo
`AUTH_ACTIVATED`	Rotas exigem login	`true`
`JWT_SECRET` , `JWT_EXPIRES_IN`	Assinatura e validade do login	gerado / `1d`
`ORCHESTRATOR_AUTO`	Distribuição automática de tarefas	`true`
`EDGE_TOKEN`	Senha da borda (a mesma vai no `.env` da borda)	gerado; obrigatório
`EDGE_TIMEOUT_MS`	Tempo máximo esperando a borda responder um comando	`3000`
`EDGE_URL`	Modo antigo (o back conecta na borda)	**vazio**, exatamente `EDGE_URL=` sem espaços depois
`VITE_MARI_NETWORK_ID`	Gravado no build do front	`0x1200`

- **No servidor só vale este arquivo.** O `Backend/server/.env` é do dev local: o Docker nem leva ele para dentro da imagem.
- **`DB_PASSWORD` fica fixa depois da primeira subida**: o Postgres grava a senha quando cria o banco. Para trocar depois, veja "Banco".
- Nunca faça commit do `.env` (já está no `.gitignore` desta pasta) nem cole os valores em chat ou issue.

Passo 8: subir tudo

```
cd ~/tcc/back/deploy
sudo docker compose up -d --build
```

A primeira vez demora 5 a 10 minutos (instala dependências e compila front e back). Depois:

```
sudo docker compose ps
```

Os três (`db` , `backend` , `web`) devem estar **running** (o `db` também **healthy**).

Abra `https://<seu DOMAIN>` . No primeiro acesso o certificado pode levar 1 a 2 minutos para sair.

Passo 9: criar o primeiro usuário

Com `AUTH_ACTIVATED=true` o front pede login, e o primeiro usuário é criado pela API:

1. Abra `https://api.<seu DOMAIN>/api` (Swagger).
2. **Auth → POST /auth/register → Try it out**, preencha `username` e `password` → **Execute**.
3. Entre no front com esse usuário.

Parte B: borda

Passo 10: preparar o computador da borda e ligar no back

A borda é quem conecta no back, então tudo deste passo acontece no **computador da borda**: qualquer máquina com Windows, Linux ou macOS, com internet e o gateway no USB. Ela só faz conexões de saída (como um navegador), então não precisa abrir porta no firewall nem no roteador.

10.1 Instalar (uma vez)

1. Instale o **Node.js 22** (com npm) e o **git**:
   - Windows: instaladores de https://nodejs.org (versão 22 LTS) e https://git-scm.com.
   - macOS: instaladores dos mesmos sites (ou `brew install node@22 git`).
   - Linux: Node 22 pelo gerenciador de pacotes da distro ou pelo `nvm`; `git` pelo gerenciador de pacotes.
2. Baixe o código, numa branch da borda que tenha o uplink (o `Edge/server/.env.example` traz `BACKEND_URL`), e instale as dependências:

```
git clone -b <branch-da-borda> https://github.com/Lucas-O-S/TCC_Swarm_Robots.git
cd TCC_Swarm_Robots/Edge/server
npm install
```

3. Crie o `.env` a partir do modelo: `cp .env.example .env` (no Prompt de Comando do Windows: `copy .env.example .env`).

10.2 Achar a porta do gateway

Ligue o nRF5340 DK pelo conector **J2** (o USB do J-Link, perto do botão de power). Ele aparece como duas ou três portas seriais; a do gateway é a **primeira (VCOM0)**.

Sistema	Como achar	Exemplo de `MARI_PORT`
Windows	**Gerenciador de Dispositivos → Portas (COM e LPT)** → "JLink CDC UART Port". Não aparece? Instale o driver J-Link da SEGGER (vem no *nRF Command Line Tools*)	`COM5`
Linux	`ls /dev/ttyACM*`	`/dev/ttyACM0`
macOS	`ls /dev/tty.usbmodem*`	`/dev/tty.usbmodem0010500667001`

No Linux, dê permissão de acesso à serial uma vez: `sudo usermod -aG dialout $USER` e saia/entre de novo na sessão.

10.3 Configurar o `.env` da borda

1. Na EC2, veja o domínio e o token:

```
grep -E '^(DOMAIN|EDGE_TOKEN)=' ~/tcc/back/deploy/.env
```

2. No **`Edge/server/.env`** do computador da borda:

```
GATEWAY_MODE=mari
MARI_PORT=<porta do 10.2>
MARI_NETWORK_ID=<id da rede gravado no gateway>
MARI_AUTO_START=true
BACKEND_URL=https://api.<seu DOMAIN>/edge
EDGE_TOKEN=<o EDGE_TOKEN da EC2>
EDGE_ID=lab-borda
EDGE_AUTH_ACTIVATED=true
```

- O `BACKEND_URL` **tem que terminar em** `/edge` . Sem isso a borda cai no socket do front e é ignorada.
- `MARI_NETWORK_ID` tem que ser o mesmo gravado no firmware do gateway.
- `MARI_AUTO_START=true` faz a borda dar o *start* nos robôs que ligam no bootloader, sem script extra.
- `EDGE_ID` é o nome da borda no log do back (vazio = nome do computador).
- `EDGE_AUTH_ACTIVATED=true` protege a API local da borda (porta 3001) na rede desse computador; o uplink não depende dele.

10.4 Subir a borda e não deixar o computador dormir

1. Com o gateway no USB, dentro de `Edge/server` :

   ```
   npm run start
   ```

   Ela reconecta sozinha se a internet ou o back caírem. (Atualizou a borda? Rode `npm install` antes.)

2. **O computador não pode dormir** enquanto o robô estiver em uso:

Sistema	Como
Windows	**Configurações → Sistema → Energia** → suspensão **Nunca** (na tomada). Ou, no PowerShell como administrador: `powercfg /change standby-timeout-ac 0`
macOS	Deixe um terminal aberto com `caffeinate -dims` (`Ctrl+C` para soltar)
Linux	Deixe um terminal aberto com `systemd-inhibit --what=sleep:idle --why=borda sleep infinity` (`Ctrl+C` para soltar)

10.5 Conferir

- Terminal da borda: `[UPLINK] conectado no back em https://api.<seu DOMAIN>/edge`
- EC2: `cd ~/tcc/back/deploy && sudo docker compose logs backend | grep EDGE` → `[EDGE] borda conectada: edgeId=lab-borda, ip=...`

> O back aceita **uma borda por vez**. Enquanto a borda estiver ligada na nuvem, não deixe um back local (de dev) conectado nela também pelo `EDGE_URL` : os dois mandariam no robô.

Parte C: GitHub (pipelines)

A verificação roda nos computadores do GitHub. O deploy roda **na própria EC2**, por um **runner self-hosted**: um programa do GitHub instalado nela que fica esperando jobs. Ele só faz conexão de saída para o GitHub: não abre porta e não precisa de chave guardada em secret.

Passo 11: instalar o runner na EC2

1. Na EC2, instale a biblioteca que o runner usa:

   ```
   sudo dnf install -y libicu
   ```

2. No GitHub: repositório → **Settings → Actions → Runners → New self-hosted runner** → **Linux**, Architecture **x64**. A página mostra dois blocos de comandos: *Download* e *Configure*.

   > **O token**: na linha `./config.sh --url ... --token XXXX` do bloco *Configure*. É um código temporário (vale cerca de 1 hora) que serve só para registrar a máquina como runner deste repositório. Cada vez que a página abre sai um novo. Não compartilhe: enquanto vale, alguém poderia registrar a máquina dele e receber os seus jobs de deploy.

3. Na EC2, vá para a pasta pessoal (`cd ~`) e cole os comandos do bloco **Download**, um por vez (começa com `mkdir actions-runner && cd actions-runner` e termina com `tar xzf ...`). Rode sem `sudo` . A conferência do hash é opcional; se der `shasum: command not found` , troque `shasum -a 256 -c` por `sha256sum -c` .

4. **Configure** copiando a linha do `./config.sh` da página e acrescentando o final, para já sair com nome e etiqueta e sem perguntas:

   ```
   ./config.sh --url https://github.com/Lucas-O-S/TCC_Swarm_Robots --token <TOKEN DA PÁGINA> --name tcc-aws --labe
   ```

   A etiqueta **`prod`** é o que os workflows de deploy procuram (`runs-on: [self-hosted, prod]`).

5. **Não** rode o `./run.sh` do fim da página (ele só funciona com o terminal aberto). Instale como serviço, ainda dentro da pasta `actions-runner` :

   ```
   sudo ./svc.sh install ec2-user
   sudo ./svc.sh start
   sudo ./svc.sh status
   ```

   O último tem que mostrar `active (running)` e, no log, `Listening for Jobs` . O serviço sobe sozinho junto com a máquina.

6. Confira em **Settings → Actions → Runners**: o runner aparece **Idle** com as etiquetas `self-hosted` , `Linux` , `X64` e `prod` .

Se o runner ficou sem a etiqueta `prod` (configurado com as perguntas e Enter): clique no runner → engrenagem ao lado de **Labels** → digite `prod` → **Create new label "prod"**. Não precisa reinstalar. Maiúscula não importa (`Prod` também serve).

`Cannot configure the runner because it is already configured` : o registro já foi feito. Se o runner aparece na lista do GitHub, só adicione a etiqueta como acima e siga para o serviço. Se não aparece, apague a configuração local e refaça o passo 4 com um token novo:

```
cd ~/actions-runner
rm -f .runner .credentials .credentials_rsaparams
```

Passo 12: proteger o Actions (repositório público)

Settings → Actions → General → Fork pull request workflows → marque **Require approval for all external contributors**. Sem isso, um PR de fora poderia tentar rodar código na sua EC2 pelo runner.

Passo 13: travar o merge quando a verificação falhar

Sem regra de proteção, um check vermelho não impede o merge. Crie uma regra para cada branch:

Branch	Checks obrigatórios
`main-Back`	`deploy` (origem tem que ser `hml-Back`) e `Verificar backend / Build e testes`
`main-Front`	`deploy` (origem tem que ser `hml-Front`) e `Verificar frontend / Lint e build`

1. Abra um PR para a branch e deixe os checks rodarem uma vez (o GitHub só lista na busca os checks que já rodaram).
2. **Settings → Branches → Add branch protection rule** (ou *Add classic branch protection rule*).
3. *Branch name pattern*: `main-Back` (depois repita para `main-Front`).
4. Marque **Require status checks to pass before merging** e, na busca, adicione os dois checks da tabela.
5. Para valer também para admins, marque **Do not allow bypassing the above settings**. **Create**.

Passo 14: variáveis (só se os caminhos forem outros)

Os deploys esperam os clones em `/home/ec2-user/tcc/back` e `/home/ec2-user/tcc/front`. Em outro lugar? Crie as variáveis `SERVER_DIR` e/ou `FRONT_SERVER_DIR` em **Settings → Secrets and variables → Actions → Variables**. Não há secrets: nada de senha fica no GitHub.

Passo 15: primeiro deploy

1. Faça um merge na `main-Back` (pelo fluxo `dev-Back → hml-Back → main-Back`) com mudança em `Backend/server/` ou `deploy/`.
2. Aba **Actions** → *Deploy Backend em Produção (EC2)*: o job **Verificar backend** roda no GitHub; depois **Subir no servidor** roda no runner e termina com ✅ `Backend no ar.`
3. Mesma coisa para o front com um merge na `main-Front` (mudança em `Frontend/`) → *Deploy Frontend em Produção (EC2)* → ✅ `Site no ar.`

Se uma execução ficar em `Waiting for a runner to pick up this job` com `Requested labels: self-hosted, prod`, o runner está desligado ou sem a etiqueta (ver "Problemas comuns"). Se a linha `Job defined at` terminar em `@refs/heads/main`, é uma versão antiga do workflow que ainda reagia à `main`: cancele (**Cancel workflow**).

Parte D: referência

Como a pipeline funciona

```
Merge/Back/<tarefa> —PR→ dev-Back —PR→ hml-Back —PR→ main-Back ———→ deploy do back na EC2
                                  origem       origem + build/test
Merge/Front/<tarefa> —PR→ dev-Front —PR→ hml-Front —PR→ main-Front —→ deploy do front na EC2
                                   origem        origem + lint/build
main-Back / main-Front / main-Edge —PR→ main   (só confere a origem; a main não faz deploy)
```

Workflow	Quando roda	O que faz
`hmlBackRule.yml`, `hmlFrontRule.yml`	PR para `hml-Back` / `hml-Front`	Confere se a origem é `dev-Back` / `dev-Front`
`mainBackRule.yml`	PR para `main-Back`	Confere a origem (`hml-Back`) e chama `backVerify.yml`
`mainFrontRule.yml`	PR para `main-Front`	Confere a origem (`hml-Front`) e chama `frontVerify.yml`
`MainToMainWorkflow.yml`	PR para `main`	Confere se a origem é `main-Back`, `main-Front` ou `main-Edge`
`backVerify.yml`	Chamado pelos outros	`npm ci`, `npm run build`, `npm test` em `Backend/server`
`frontVerify.yml`	Chamado pelos outros	`npm ci`, `npm run lint`, `npm run build` em `Frontend`
`prodBackDeploy.yml`	Merge na `main-Back` com mudança em `Backend/server/` ou `deploy/`	Verifica; no runner, põe `~/tcc/back` **no commit verificado** e reconstrói **só o `backend`**; confere se a API respondeu
`prodFrontDeploy.yml`	Merge na `main-Front` com mudança em `Frontend/`	Verifica; no runner, põe `~/tcc/front` **no commit verificado** e reconstrói **só o `web`**; confere se o site respondeu

- Se a verificação falhar, nada vai para o servidor. Se o build falhar no servidor, o container antigo continua no ar.
- Cada workflow vive na sua branch: os do back chegam pelas branches do back, os do front pelas do front.
- **Rodar na mão**: **Actions** → o workflow → **Run workflow**, escolhendo em *Use workflow from* a `main-Back` (back) ou a `main-Front` (front); de outra branch ele não faz nada. O botão só aparece quando o arquivo também existe na `main` (a branch padrão do repositório).
- **Learner Lab**: com a EC2 desligada o job de deploy fica na fila e roda quando você liga a máquina. Se passar 24 h na fila, vence; aí rode na mão.

Dia a dia no Learner Lab

Ligar (a cada sessão nova):

1. **Start Lab** → bolinha verde → **AWS**.
2. **EC2 → Instâncias →** **`tcc-server`** **→ Estado da instância → Iniciar instância**.
3. Espere 1 a 2 minutos. Docker, containers e runner sobem sozinhos; o site volta no mesmo endereço.
4. No computador da borda: gateway no USB, borda rodando (`npm run start`) e o computador sem dormir (passo 10.4). Se a borda já estava ligada, ela reconecta sozinha quando o back voltar.

Desligar: o lab desliga sozinho no fim da sessão. Para economizar crédito antes disso: **Estado da instância → Interromper instância**. Nunca *Encerrar*.

Atualizar na mão (sem pipeline)

```
cd ~/tcc/back  && git pull
cd ~/tcc/front && git pull
cd ~/tcc/back/deploy && sudo docker compose up -d --build
```

Comandos úteis (dentro de `~/tcc/back/deploy`)

O quê	Comando
Ver o estado	`sudo docker compose ps`
Logs do back ao vivo (`Ctrl+C` sai)	`sudo docker compose logs -f backend`
Conexão da borda	`sudo docker compose logs backend | grep EDGE`
Logs do Caddy/HTTPS	`sudo docker compose logs web`
Reiniciar só o back	`sudo docker compose restart backend`
Mudou o `.env`	`sudo docker compose up -d`
Mudou `DOMAIN` ou `VITE_*` (gravados no build do front)	`sudo docker compose up -d --build`
Parar tudo (mantém os dados)	`sudo docker compose down`
Estado do runner	`cd ~/actions-runner && sudo ./svc.sh status`

Banco

- Os dados ficam no volume `tcc_db_data` e sobrevivem a restart, rebuild e `down`.
- O `init.sql` **só roda na primeira subida** (volume vazio). Mudança de schema depois disso é SQL manual:

  ```
  sudo docker compose exec -T db psql -U adm -d mari_database < ../Backend/server/database/sql/<arquivo>.sql
  ```

- Backup: `sudo docker compose exec -T db pg_dump -U adm mari_database > ~/backup.sql`
- Trocar a senha do banco depois da primeira subida (mudar só o `.env` faz o back parar de conectar):

  ```
  sudo docker compose exec db psql -U adm -d mari_database -c "ALTER USER adm PASSWORD 'NOVA_SENHA';"
  ```

  Depois ponha a mesma senha em `DB_PASSWORD` no `.env` e rode `sudo docker compose up -d`.
- Zerar o banco (**apaga tudo**) e recriar pelo `init.sql`:

  ```
  sudo docker compose down
  sudo docker volume rm tcc_db_data
  sudo docker compose up -d
  ```

Problemas comuns

Servidor

Sintoma	Causa provável	O que fazer
Site não abre: **tempo esgotado** (timeout)	Instância parada ou portas 80/443 fechadas	Instância **Executando**? Grupo de segurança com HTTP 80 e HTTPS 443 de `0.0.0.0/0`
Site não abre: **recusou a conexão**	Nada escutando: os containers não subiram	`sudo docker compose ps` ; se faltar o `web` , `sudo docker compose up -d --build` e ler o erro
Erro de certificado nos primeiros minutos	Caddy ainda tirando o certificado	Esperar 1 a 2 min; ver `sudo docker compose logs web`
Front abre, mas login/robôs dão erro de rede	`DOMAIN` errado (front compilado com outro IP)	`DOMAIN` = IP elástico com traços; depois `sudo docker compose up -d --build`
O endereço mudou	IP elástico não associado	Passo 3; corrigir `DOMAIN` no `.env` e `sudo docker compose up -d --build`
`compose build requires buildx 0.17.0 or later`	Buildx antigo do Amazon Linux	Rodar de novo o `setup-ec2.sh` da versão atual
`required variable EDGE_TOKEN is missing a value` (ou `DOMAIN` , `DB_PASSWORD` ...)	Variável vazia no `.env`	Preencher no `deploy/.env` (passo 7)
Build do `web` falha com `error TS...`	Erro de tipo no front (`npm run build` roda `tsc -b` antes do Vite)	Corrigir no código, conferir com `npm run build` no seu computador e subir pelo fluxo
Build morre com `Killed` / código 137	Falta de memória	`free -h` deve mostrar swap de 2G; se mostrar, troque para `t3.medium` (instância parada → Ações → Configurações da instância → Alterar tipo de instância)
Back não conecta no banco depois de mudar `DB_PASSWORD`	O Postgres guardou a senha antiga	Ver "Banco" (`ALTER USER`)
Instance Connect não conecta	Porta 22 fechada	Regra SSH 22 de entrada, ou use o Session Manager
`git pull` pede senha toda vez	Repo privado	Usar o token como senha; ou `git config --global credential.helper store` (grava o token em texto puro na EC2)

Borda

Sintoma	Causa provável	O que fazer
Comandos ao robô respondem **503** ("Nenhuma borda conectada")	A borda não está conectada no back	Computador da borda acordado e com internet? Borda rodando? `BACKEND_URL` termina em `/edge` ? Ver o log `EDGE` do back e o `[UPLINK]` da borda
Borda: `back recusou a conexão: token inválido` / back: `borda recusada: token inválido`	`EDGE_TOKEN` diferente entre a EC2 e a borda	Mesmo valor nos dois `.env` ; a borda tenta de novo sozinha a cada 30 s (ou reinicie)
Back: `borda recusada: ... já está conectada`	Outra borda (outro `EDGE_ID`) já está ligada nesse back	Desligar a outra; o back aceita uma borda por vez
Borda: `não conectou no back`	Back ou `web` fora do ar, ou `BACKEND_URL` errado	`sudo docker compose ps` na EC2; conferir o `BACKEND_URL`
Comandos respondem **502**	A borda não respondeu a tempo ou deu erro interno	Ver o terminal da borda; conferir o gateway no USB
Borda não abre a serial (`No such file` , `File not found` , `cannot open`)	`MARI_PORT` errada ou gateway desconectado	Passo 10.2; no Windows a porta é `COMx` , no Linux `/dev/ttyACMx`
Linux: `Permission denied` na serial	Usuário sem acesso à serial	`sudo usermod –aG dialout $USER` e entrar de novo na sessão
Windows: o DK não aparece em "Portas (COM e LPT)"	Falta o driver J-Link	Instalar o *nRF Command Line Tools* (traz o driver da SEGGER) e religar o cabo
Borda conectada, mas nenhum robô aparece	Porta que não é a VCOM0, `MARI_NETWORK_ID` diferente do gateway, ou robô desligado	Trocar para a outra porta do DK; conferir o id da rede; ligar o robô

GitHub / runner

Sintoma	Causa provável	O que fazer
Job parado em `Waiting for a runner to pick up this job` (`Requested labels: self-hosted, prod`)	Nenhum runner online com a etiqueta `prod`	**Settings → Actions → Runners**: sem runner → passo 11; *Offline* → ligar a EC2 / `sudo ./svc.sh start` ; *Idle* sem `prod` → adicionar a etiqueta
`Job defined at ...@refs/heads/main`	Versão antiga do workflow (reagia à `main`)	Cancelar a execução
`Libicu's dependencies is missing`	Falta a biblioteca	`sudo dnf install -y libicu` e repetir o `config.sh`
`Must not run with sudo`	`config.sh` rodado com `sudo`	Rodar sem `sudo`
Token inválido / expirado no `config.sh`	O token vale ~1 h e muda a cada abertura da página	Abrir de novo **New self-hosted runner** e usar o token novo
`Cannot configure the runner because it is already configured`	Já registrado	Passo 11, parágrafo "already configured"
Runner some quando fecha o terminal	Rodando com `./run.sh`	Instalar como serviço (`svc.sh` , passo 11.5)
Deploy falha em "Atualizar o servidor para a main-Back" (ou do front)	Arquivo versionado editado direto no clone do servidor	`cd ~/tcc/back && git status` (ou `~/tcc/front`); desfazer com `git checkout -- <arquivo>` e rodar de novo
Deploy falha em "Conferir se o backend respondeu" / "o site respondeu"	O container subiu mas não respondeu	O próprio job mostra os últimos logs; corrigir e fazer novo merge
Check vermelho mas o merge continua liberado	Falta a regra de proteção da branch	Passo 13

Segurança (o mínimo)

- `deploy/.env` tem senhas: **nunca** faça commit nem cole em chat. Vazou o `EDGE_TOKEN` ? Gere outro (`openssl rand -hex 24` , na EC2) e troque nos dois `.env` (EC2 e borda).
- O Postgres não tem porta aberta para a internet; só o `backend` enxerga.
- O computador da borda não recebe conexões de fora: é a borda que conecta no back, por HTTPS, com o `EDGE_TOKEN` . O back aceita uma borda por vez e registra no log o `edgeId` e o IP de cada conexão. Quem tiver o token consegue se passar pela borda: trate como senha.
- Mantenha `AUTH_ACTIVATED=true` . Atenção: `/auth/register` é público, então quem tiver o endereço consegue criar conta (fechar isso exige mudar o back).
- Token do runner: só registra máquina e vence em ~1 h, mas não compartilhe. Runner desconhecido na lista? Remova.
- Repositório público: mantenha a aprovação de PRs de fora (passo 12).
- Opcional: restringir o SSH ao Instance Connect. Na regra SSH 22, troque a origem por **Personalizado** e escolha a lista `com.amazonaws.us-east-1.ec2-instance-connect` .

Outro ambiente (ex.: homologação)

Para um segundo ambiente, repita as Partes A e B em uma **EC2 nova**, mudando:

- **Nome** `tcc-hml` e **IP elástico novo** (o domínio sai diferente sozinho).

- **Clones** nas branches do ambiente (ex.: `hml-Back` e `hml-Front`).
- **`.env` próprio**, com `EDGE_TOKEN` diferente do de produção: assim uma borda de um ambiente nunca entra no outro.
- **Borda**: o back aceita uma borda por vez e só existe um gateway. Para homologação, rode uma segunda borda (em qualquer computador, ou numa cópia da pasta `Edge` no mesmo), só com robô simulado: `GATEWAY_MODE=simulator`, `SIMULATOR_FAKE_ADVERTISEMENT=true`, `EDGE_PORT=3002` e `BACKEND_URL=https://api.<domínio-hml>/edge`.
- **Runner**: registre com **outra etiqueta** (ex.: `--labels hml`), **nunca `prod`**, senão os deploys de produção podem cair nessa máquina.
- **Pipeline**: os workflows de deploy de hoje atendem só produção (`main-Back` / `main-Front` + etiqueta `prod`). Até eles serem adaptados para receber branch e etiqueta, atualize o outro ambiente na mão ("Atualizar na mão").